AI-Powered Real-Time Battery Fitness Assessment Platform

MACRO PROJECT-I

Milestone 1 — Foundation & Technical Proof of Concept

Submitted By: -

Dhruv Urmaliya (BTAI25O1052)

Aakanksha Vidushi Pandey (BTAI25O1001)

Under the Guidance of: -

Dr. Neelam Arya



CENTRE FOR ARTIFICIAL INTELLIGENCE

Madhav Institute of Technology & Science

Gwalior - 474005 (MP) est. 1957

July - Dec 2026

1. Problem Statement

Right now, fitness assessment in most sports training setups relies entirely on a coach's subjective judgment. Two athletes doing the same set of sit-ups often get scored differently depending on which coach is watching, what mood they're in, or how clearly they can see the movement from their angle. There's no standard, repeatable, objective measurement.
A webcam captures exactly what the coach sees. The problem isn't the visual data—it's that we don't have a system that turns that video into real, objective numbers that both the coach and athlete can trust.

2. Existing Approaches and Their Limitations

Approach	Limitation
Manual observation	Depends on the coach's experience and judgment. Two coaches can give different scores for the same performance.
Wearables (IMU / heart-rate)	Give accurate data, but every athlete needs their own device. Too expensive for teams with many athletes.
Existing Mobile apps	Most only count repetitions. They don't check if the form is correct, and they don't explain why a rep was rejected.

3. Our Solution

We are building a system that works with just a standard webcam. No wearables, no special equipment.
The idea is simple:

- The webcam records the athlete in real time.
- An AI model (YOLO26-Pose) detects 17 key body points (shoulders, elbows, knees, etc.) in every frame.
- Our code calculates joint angles from those points.
- Based on those angles, we detect whether the athlete is in the "up" or "down" position, count valid repetitions, and give a form score.

All this happens live, and the athlete or coach sees the feedback on screen immediately.

4. Objectives

Our main goal for this milestone was to test whether the basic pipeline actually works. Specifically, we wanted to:

- Capture live video from a webcam.
- Detect human pose in real time using YOLO26-Pose.
- Extract 17 key points from each frame, and ignore any detections with low confidence (below 0.5).
- Calculate joint angles from those key points using basic vector math.

- Use those angles to figure out the movement state (up or down).
- Count valid repetitions and identify incorrect form.
- Prepare the groundwork for storing session data and showing it on a dashboard.

5. Scope: Fitness Tests & Parameters

Phase 1– Current Focus:
We are starting with three basic assessments: **Sit-ups, Sit & Reach, and Vertical Jump.**
Phase 2 – Future Expansion:
Broad Jump, 30m Sprint, Shuttle Run, Medicine Ball Throw, Endurance Run, Height, and Weight
Why we chose These First:
All three Phase 1 assessments involve **body posture and movement analysis using pose keypoints**. Once we establish and validate the pose → keypoint → angle/movement interpretation pipeline on these assessments, we'll move to the remaining tests, which may additionally require distance, timing, or other assessment-specific measurements—a more complex problem we'll tackle later.

6. System Architecture

Our pipeline is linear and straightforward:
Live Webcam → OpenCV → Video Frames → YOLO26-Pose → 17 Body Keypoints → Movement Analysis

The Analysis Engine has three parallel branches:

- **Repetition Counting** (tracking up/down transitions)
- **Form Analysis** (checking angles against ideal ranges)
- **Metrics Computation** (range of motion, speed, etc.)

All three converge into a **Scoring Engine**, which then saves the results and pushes them to the **Dashboard**.

Note: We're using WebSockets for real-time communication between the backend and frontend. This is separate from the AI pipeline—it's just how we send live feedback to the browser without delay.

7. Technology Stock & Why we chose it

Layer	Choice	Justification
Pose Estimation	YOLO26-Pose (yolo26n-pose.pt)	The nano version is much faster. We accepted a tiny drop in peak accuracy to get the low latency needed for live feedback.
Video I/O	OpenCV	Gives us direct access to raw frames. We needed this level of control to pre-process before passing to the model.

Backend	Flask	Lightweight and allows fast iteration while we're still heavily developing the processing pipeline.
Real-time communication	WebSocket	HTTP polling is too slow for continuous feedback. WebSockets keep a persistent connection open.
Database	PostgreSQL	Relational structure fits our data perfectly (athletes, sessions, scores).
Frontend	HTML / CSS / JavaScript	Gives us full control to render the live skeleton overlay directly in the browser.

8. Implementation Progress (What we Actually Built)

This section covers our real, hands-on work. We didn't just theorize—we coded, broke things, and fixed them.

8.1 Webcam Capture

We started by simply getting the webcam feed to display.

We hit a wall right away: the first time we ran the capture loop, the screen stayed completely black.

Instead of randomly tweaking settings, we paused and inspected the raw frame data:

What We Checked	What We Found
ret value	True — so the camera was sending data.
frame.shape	(480, 640, 3) — valid dimensions.
frame.mean()	≈ 128.x — the pixel values were normal, not zeros.

This proved the camera pipeline was working fine. The issue was purely on the display side—once we fixed that, we got a live feed.

Status: Fully working.

8.2 YOLO26-Pose Integration & Key point Extraction

Next, we integrated the AI model.

We load it with model = YOLO("yolo26n-pose.pt"). On every frame, we call model(frame), which returns a Results object.

Inside that object, results[0].keypoints.data holds a tensor for every detected person. Each person has 17 keypoints in the standard COCO format: (x, y, confidence).

One thing we learned quickly: we **must** filter out keypoints with confidence below 0.5. Without this filter, occluded or poorly lit joints would feed garbage coordinates into our angle math and break everything.

- Sample reading from Left Shoulder (index 5): (x, y, confidence) = (312.5, 244.3, 0.95)

Status: Detection is running live on webcam frames.

8.3 Custom Joint-Angle Algorithm

This was the core math part. YOLO gives us points, but it doesn't understand "bent" or "straight." We had to calculate that ourselves.

To find the angle at a joint b (say, the elbow), we take its two neighbors a (shoulder) and c (wrist). We calculate the vectors BA = a - b and BC = c - b. Then we use the standard dot-product formula:

$\theta = \arccos((BA \cdot BC) / (|BA| \times |BC|))$

We first validated this on the shoulder–elbow–wrist triplet (elbow angle). It worked perfectly. Now we're extending it to the shoulder–hip–knee triplet—which is what we actually need for detecting sit-up states.

Status: Arm angle is validated. Torso angle (for sit-ups) is currently in progress.

9. Database & UI Design (Planned)

We've already sketched out the database structure on paper. The core entities are: **Athlete**, **Session**, **Assessment**, and **Score**.

For the UI, we have a rough wireframe flow:

Login → Select Fitness Test → Live Camera with Skeleton Overlay → Results Dashboard.

Status: At the design stage. Full implementation will begin in Milestone 4.

10. Testing Status Summary

Test	Expected Result	Status
Webcam opens	Camera stream visible	Done
Frame capture	Valid frame returned	Done
Frame dimensions	640 × 480 × 3	Done
OpenCV display	Live video rendered	Done
Ultralytics import	Successful	Done
YOLO26-Pose model load	Successful	Done
Pose detection on live frame	Keypoints returned	Done
Custom angle calculation	Correct angle output	In progress
Sit-up state detection	Up/down state identified	Pending

11. Challenges we ran into

- **Confidence drops in poor lighting or when body parts are occluded.** We handled this with a hard threshold (reject anything below 0.5).
- **Frame inference speed vs. real-time requirement.** The larger YOLO models were too slow. Switching to the nano variant (yolo26n-pose.pt) brought us down to ~100-150ms per frame, which is acceptable for live feedback.

- **Webcam index handling** varies across machines. Camera 0 works on some, Camera 1 on others. We need to implement a fallback / selection dialog in the future.

12. Next Milestone Plan – Milestone 2 (27 Aug)

- Complete the torso-angle (shoulder–hip–knee) based up/down state detection for sit-ups.
- Build the repetition counting logic by tracking up/down transitions.
- Extend the validated pipeline to push-ups and squats (just adjust the joint triplets and angle ranges).
- Store the first set of real session data into the database.